ITA06 - MACHINE LEARNING

ASSIGNMENT 2 – CO2

Name : Tamil Elakiya S

Reg No : 192524443

CO2 Coverage: Neural network representation; perceptrons and perceptron learning algorithm; multilayer perceptrons; introduction to deep neural networks; backpropagation; overfitting and regularization; hyperparameter tuning; optimization strategies.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Q1 – Design a Neural Network Representation for a Classification Problem 6 Marks

Problem Context – Manufacturing Quality Inspection

A manufacturing unit records three measurements for each component: Surface Roughness | Dimensional Deviation | Vibration Level. Each component is labelled Acceptable (0) or Defective (1). The unit proposes a neural network to support quality inspection.

Your Challenge

Design a neural network representation for this binary classification problem. Determine:

- The input features and their numerical representation.
- Any preprocessing required and why it is necessary.
- Two candidate architectures: one without a hidden layer and one with a hidden layer.
- The number of input and output neurons, suitable activation functions and an output decision rule.
- The number of trainable weights and biases in each architecture.

Draw both architectures and formulate the mathematical computation from input to output. For one architecture, assign small numerical values to the inputs, weights and biases. Manually calculate the forward pass and resulting classification.

Design Inquiry

The inspection team suspects that a component becomes defective only when certain measurements interact. Explain how this requirement influences your architecture choice and identify what evidence would be needed to justify using a hidden layer.

Evidence Expected

Two labelled network diagrams, justified representation and activation choices, correct parameter counts, a worked forward pass and a reasoned architecture recommendation.

Q1 **6 Marks**

1. Input features and numerical representation

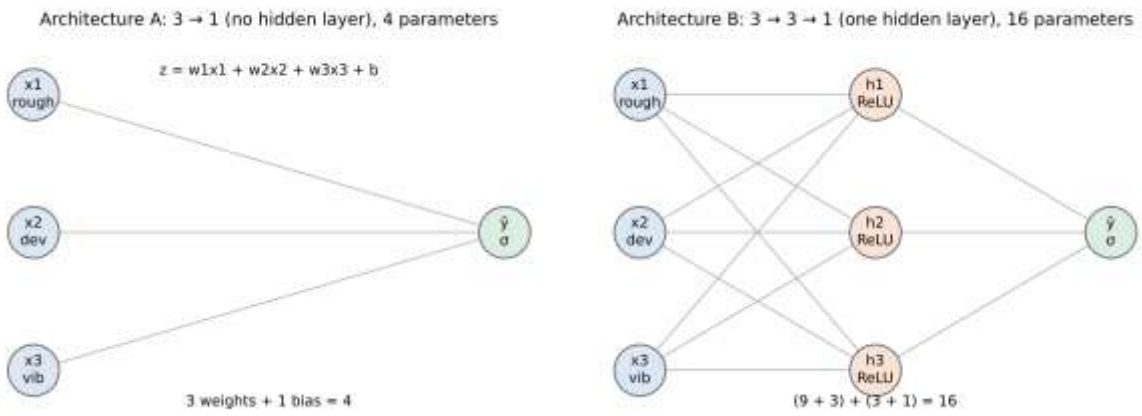
Input	Quantity	Representation	Example (raw → standardised)
x1	Surface roughness (e.g. Ra in µm)	Real number, standardised $z = (x - \mu) / \sigma$	1.8 µm → 0.5 ($\mu = 1.6$, $\sigma = 0.4$)
x2	Dimensional deviation (mm)	Real number (use the absolute deviation if the sign carries no meaning), standardised	0.09 mm → 2.0 ($\mu = 0.05$, $\sigma = 0.02$)
x3	Vibration level (mm/s RMS)	Real number, standardised	3.3 → 1.0 ($\mu = 2.5$, $\sigma = 0.8$)
y	Label	0 = Acceptable, 1 = Defective (single binary target)	–

2. Preprocessing and why it is necessary

- **Standardisation** (mean 0, standard deviation 1, with µ and σ computed on the training set only and reused for validation, test and live data). The three measurements have different units and ranges (0.09 mm versus 3.3 mm/s); without scaling, large-valued inputs dominate the weighted sum, make gradient descent zig-zag and push neurons into saturation.
- **Cleaning:** impute or reject missing readings, inspect outliers (sensor faults) before scaling because they distort µ and σ.
- **Class balance:** defective parts are usually rare, so I would stratify the splits and, if needed, weight the defective class so that missed defects are penalised more.
- **Split discipline:** train / validation / test, with the test set used once, at the end.

3. Two candidate architectures

Figure 1. Candidate architectures for quality inspection



	Architecture A (no hidden layer)	Architecture B (one hidden layer)
Input neurons	3	3
Hidden neurons	none	3, ReLU
Output neurons	1, sigmoid (probability of Defective)	1, sigmoid
Computation	$z = w_1x_1 + w_2x_2 + w_3x_3 + b$; $p = \sigma(z) = 1 / (1 + e^{-z})$	$h = \mathrm{ReLU}(W_1^T x + b_1)$; $z_2 = w_2^T h + b_2$; $p = \sigma(z_2)$
Trainable weights	3	9 (input→hidden) + 3 (hidden→output) = 12
Trainable biases	1	3 + 1 = 4
Total parameters	**4**	**16**
Decision rule	Defective (1) if $p \geq 0.5$, otherwise Acceptable (0); the threshold can be lowered (for example 0.3) if missing a defect is costlier than a false alarm	Same

Activation choices. The output neuron uses the sigmoid because it maps the weighted sum to (0, 1), interpretable as P(Defective), and pairs with binary cross-entropy to give the simple output error $p - y$. The hidden neurons use ReLU because it is cheap, does not saturate for positive inputs (reducing vanishing gradients) and introduces the non-linearity needed for interactions; tanh would also be acceptable.

4. Worked forward pass (Architecture B)

Raw component: roughness 1.8, deviation 0.09, vibration 3.3 → standardised input **x = (0.5, 2.0, 1.0)**. Chosen small parameters: W_1 (rows = inputs, columns = hidden neurons) = [[0.4, −0.2, 0.1], [0.3, 0.5, −0.4], [−0.1, 0.2, 0.6]], b_1 = (0.1, −0.1, 0.0), w_2 = (0.5, −0.3, 0.8), b_2 = −0.2.

Step	Calculation	Result
z_{11}	0.5(0.4) + 2.0(0.3) + 1.0(−0.1) + 0.1	0.8000 → h_1 = ReLU = 0.8000
z_{12}	0.5(−0.2) + 2.0(0.5) + 1.0(0.2) − 0.1	1.0000 → h_2 = 1.0000
z_{13}	0.5(0.1) + 2.0(−0.4) + 1.0(0.6) + 0.0	−0.1500 → h_3 = ReLU = 0.0000
z_2	0.5(0.8) + (−0.3)(1.0) + 0.8(0.0) − 0.2	−0.1000
$p = \sigma(z_2)$	1 / (1 + e^0.1)	0.4750
Decision	0.4750 < 0.5	**Class 0 – Acceptable**

(For comparison, Architecture A with w = (0.6, 0.4, 0.5), b = −1 gives z = 0.6, p = 0.6457, i.e. Defective: different arbitrary weights give different outputs, which is why the weights must be learned.)

5. Design inquiry: interacting measurements

If a component is defective **only when certain measurements interact** (for example roughness matters only when vibration is high), the log-odds can no longer be written as a sum of separate contributions of each measurement. Architecture A is a single linear boundary (logistic regression): it can add up evidence but cannot represent "high roughness AND high vibration, but not either alone" in a curved or two-region form, and cannot represent exclusive-or–type interactions at all (Q2 proves this for two inputs). The hidden layer of Architecture B creates new features (h_1, h_2, h_3) that are non-linear combinations of the inputs, so the output neuron can separate classes with a non-linear boundary. The suspicion of interaction therefore **favours B**, but a hidden layer costs more parameters, data and interpretability, so I would recommend building A first as the baseline and adopting B only if the evidence below supports it.

Evidence needed to justify the hidden layer:

- Validation loss of B lower than that of A by more than the seed-to-seed spread, over at least three seeds, with the same data partitions;
- Residual analysis of A: errors concentrated in particular combinations of two measurements (for example high roughness with high vibration), shown in two-dimensional plots of misclassified parts;
- Improvement of A after adding explicit product features ($x_1 \cdot x_3$, …), which confirms an interaction, and a curved boundary in partial-dependence plots;
- Enough data (rule of thumb: at least 10–20 labelled parts per parameter, here 160–320 for 16 parameters) and no sign that B overfits (small train–validation gap).

Recommendation: Architecture B (3–3–1, ReLU/sigmoid, 16 parameters) as the working design, because the interaction suspicion is plausible and the model is tiny; keep A as the interpretable baseline that B must beat.

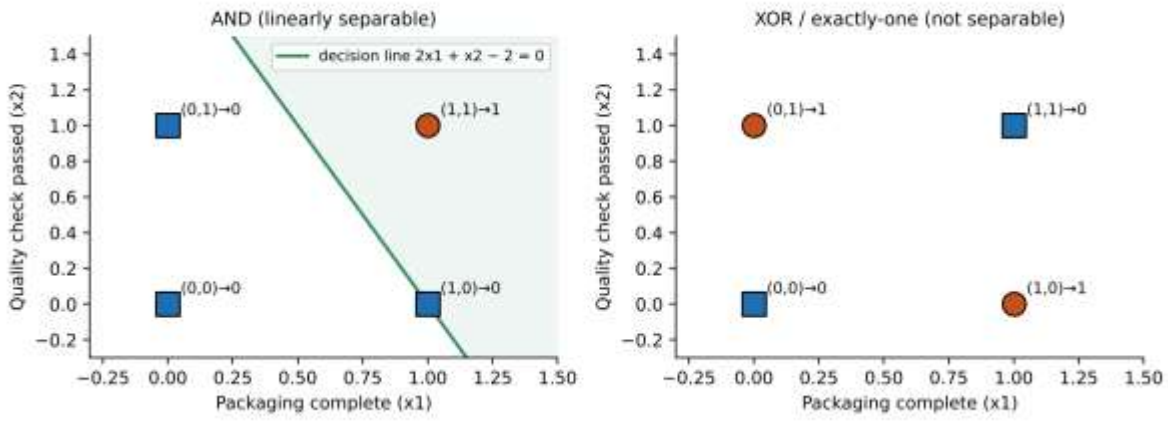
Q2 – Investigate the Learning Behaviour and Limitations of a Perceptron 4 Marks

1. Input–output tables and prediction

Packaging complete (x1)	Quality check passed (x2)	AND: dispatch (target)	Exactly one (review flag)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Prediction: a single perceptron **can** learn the dispatch (AND) rule because the positive point (1,1) can be separated from the other three by one straight line (for example $2x_1 + x_2 - 2 = 0$). It **cannot** learn the exactly-one rule, because the two positive points (0,1) and (1,0) lie on one diagonal and the two negative points (0,0) and (1,1) on the other: no straight line separates them.

Figure 2. Geometry of the two dispatch decisions (blue square = 0, orange circle = 1)



Algebraic confirmation for the exactly-one rule: a perceptron needs $b \leq 0$, $w_1 + w_2 + b \leq 0$ (negatives) and $w_1 + b > 0$, $w_2 + b > 0$ (positives). Adding the last two gives $w_1 + w_2 + 2b > 0$, hence $w_1 + w_2 + b > -b \geq 0$, which contradicts $w_1 + w_2 + b \leq 0$. No weights exist.

2. Perceptron specification

Item	Value
Model	$\hat{y} = 1$ if $z = w_1x_1 + w_2x_2 + b > 0$, otherwise 0 (step function, threshold 0)
Initial weights and bias	w = (0, 0), b = 0
Learning rate	$\eta = 1$ for the dispatch rule (exact integer arithmetic); $\eta = 0.01$, 0.1 and 1 for the review flag
Update rule	error $e = \text{target} - \hat{y}$; $w_i \leftarrow w_i + \eta \cdot e \cdot x_i$; $b \leftarrow b + \eta \cdot e$
Maximum epochs	100 (stop earlier if an epoch has no misclassification)
Presentation order	(0,0), (0,1), (1,0), (1,1) every epoch

3. Dispatch rule (AND): predictions, first-epoch updates, training record

Predictions before training (w = (0,0), b = 0 → z = 0 → $\hat{y}$ = 0 for all inputs): 0, 0, 0, 0 versus targets 0, 0, 0, 1 → 1 misclassification.

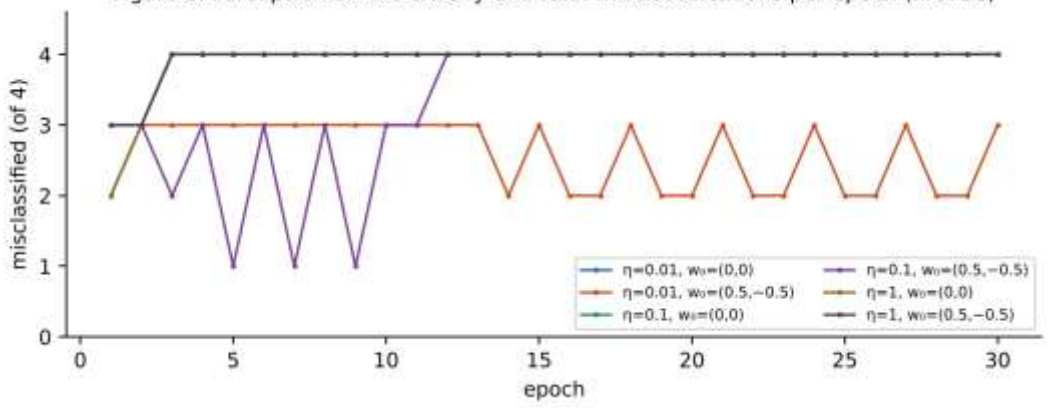
Input	w, b before	Target	$\hat{y}$	Error	w, b after
(0,0)	(0, 0), 0	0	0	0	(0, 0), 0
(0,1)	(0, 0), 0	0	0	0	(0, 0), 0
(1,0)	(0, 0), 0	0	0	0	(0, 0), 0
(1,1)	(0, 0), 0	1	0	+1	(1, 1), 1

Misclassifications per epoch: 1, 3, 3, 2, 1, 0 (epochs 1 to 6). Only the error on (1,1) changes the parameters in epoch 1: w becomes (1, 1) and b becomes 1. **Final parameters** after epoch 6: w = (2, 1), b = -2. **Final predictions:** 0, 0, 0, 1 – all four correct, as the perceptron convergence theorem guarantees for separable data. (Final boundary $2x_1 + x_2 - 2 = 0$ is drawn in Figure 2.)

4. Changed requirement: exactly one indicator = 1

η	Initial w, b	Misclassified per epoch (epochs 1–8)	Minimum	Epoch 100	Converged?
0.01	(0, 0), 0	2, 3, 4, 4, 4, 4, 4, 4	2	4	No (100 epochs)
0.1	(0, 0), 0	2, 3, 4, 4, 4, 4, 4, 4	2	4	No (100 epochs)
1	(0, 0), 0	2, 3, 4, 4, 4, 4, 4, 4	2	4	No (100 epochs)
0.01	(0.5, −0.5), 0.25	3, 3, 3, 3, 3, 3, 3, 3	1	3	No (100 epochs)
0.1	(0.5, −0.5), 0.25	3, 3, 2, 3, 1, 3, 1, 3	1	4	No (100 epochs)
1	(0.5, −0.5), 0.25	3, 3, 4, 4, 4, 4, 4, 4	3	4	No (100 epochs)

Figure 3. Perceptron on the exactly-one rule: misclassifications per epoch (first 30)



First epoch for $\eta = 1$, $w = (0,0)$, $b = 0$: (0,0) correct; (0,1) target 1, $\hat{y} = 0 \rightarrow w = (0,1)$, $b = 1$; (1,0) target 1, $\hat{y} = 1$ correct; (1,1) target 0, $\hat{y} = 1 \rightarrow w = (-1, 0)$, $b = 0$. Predictions before training were 0,0,0,0 (2 errors). After epoch 100 with zero initial weights the predictions are 1, 1, 0, 0 (2 wrong); the misclassification count never reaches 0 in any of the six runs.

Does changing the learning rate resolve the difficulty? No. With zero initial weights the three learning rates give identical misclassification sequences (2, 3, 4, 4, …): η only rescales w and b, and the sign of z does not change. With a non-zero start the sequences differ (they oscillate between 1 and 4 errors) but never reach 0. This is a **limitation of the model representation**, not a **training-configuration problem**: a training-configuration problem (learning rate too small or too large, too few epochs, poor initialisation) can be cured by changing those settings and shows up as slow or unstable but eventually successful learning; here the algebraic proof above shows that no weight vector exists, so no setting of η, initial weights or epochs can succeed. The remedy is to change the representation (add a hidden layer or an engineered feature such as $x_1 \cdot x_2$), which is the subject of Q3.

Q3 – Develop and Verify Backpropagation for a Multilayer Perceptron 6 Marks

1. Network, forward pass and loss

Architecture $2 \rightarrow 2 \rightarrow 1$ with sigmoid $\sigma(z) = 1/(1 + e^{-z})$ in both layers. For input row x (1×2): $\mathbf{z_1 = xW_1 + b_1}$, $\mathbf{h = \sigma(z_1)}$, $\mathbf{z_2 = hW_2 + b_2}$, $\mathbf{a = \sigma(z_2)}$ (W_1 is 2×2, W_2 is 2×1). Loss for one example (averaged over a batch): $\mathbf{L = -[y \ln a + (1 - y) \ln(1 - a)]}$.

2. Gradient derivation

Use $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. **Output layer:** $\partial L/\partial a = -y/a + (1 - y)/(1 - a) = (a - y)/[a(1 - a)]$ and $\partial a/\partial z_2 = a(1 - a)$, so $\boldsymbol{\delta_2 = \partial L/\partial z_2 = a - y}$. Then $\partial L/\partial W_2 = h^T \delta_2$ and $\partial L/\partial b_2 = \delta_2$. **Hidden layer:** the error is sent back through W_2, $\partial L/\partial h = \delta_2 W_2^T$, and through the sigmoid, $\boldsymbol{\delta_1 = \partial L/\partial z_1 = (\delta_2 W_2^T) \odot h \odot (1 - h)}$. Then $\partial L/\partial W_1 = x^T \delta_1$ and $\partial L/\partial b_1 = \delta_1$. For a batch of m examples the gradients are averaged (divide by m).

3. Update equations

With learning rate η: $\mathbf{W_2 \leftarrow W_2 - \eta\, h^T \delta_2}$, $\mathbf{b_2 \leftarrow b_2 - \eta\, \delta_2}$, $\mathbf{W_1 \leftarrow W_1 - \eta\, x^T \delta_1}$, $\mathbf{b_1 \leftarrow b_1 - \eta\, \delta_1}$.

4. Why the hidden activation must be non-linear

If the hidden activation were linear (identity), the output would be $z_2 = (xW_1 + b_1)W_2 + b_2 = x(W_1W_2) + (b_1W_2 + b_2)$, which is again a linear function of x: the whole network would collapse to a single perceptron and could not solve the exactly-one rule (Q2). A non-linear hidden activation lets the two hidden neurons bend the input space (for example one detects "x_1 or x_2" and the other "x_1 and x_2"), so that the output neuron can separate the classes with a linear cut in hidden space. Differentiability (sigmoid) is also needed so that gradients can flow back.

5. Worked numerical training step

Initial parameters: $W_1 = [[0.5, -0.3], [0.2, 0.4]]$ (row 1 = weights from x_1), $b_1 = (0.1, -0.1)$, $W_2 = (0.6, -0.4)^T$, $b_2 = 0.2$. Training example $x = (1, 0)$, $y = 1$ (exactly-one rule), $\eta = 0.5$.

Quantity	Calculation	Value
z_1	$x\,W_1 + b_1 = (0.5 + 0.1, -0.3 - 0.1)$	(0.6000, −0.4000)
$h = \sigma(z_1)$	$\sigma(0.6), \sigma(-0.4)$	(0.6457, 0.4013)
z_2	$0.6457(0.6) + 0.4013(-0.4) + 0.2$	0.4269
$a = \sigma(z_2)$	$\sigma(0.4269)$	0.6051
Loss L	$-\ln(0.6051)$	0.5023
δ_2	$a - y = 0.6051 - 1$	−0.3949
$\partial L/\partial W_2$, $\partial L/\partial b_2$	$h^T\delta_2$; δ_2	(−0.2550, −0.1585) ; −0.3949
δ_1	$(\delta_2 W_2^T) \odot h(1-h) = (-0.2369 \times 0.2288, 0.1580 \times 0.2403)$	(−0.0542, 0.0379)
$\partial L/\partial W_1$, $\partial L/\partial b_1$	$x^T\delta_1$ (row 2 = 0 because $x_2 = 0$) ; δ_1	[[−0.0542, 0.0379], [0, 0]] ; (−0.0542, 0.0379)
Updated W_2, b_2	$0.6 + 0.5(0.2550), -0.4 + 0.5(0.1585), 0.2 + 0.5(0.3949)$	(0.7275, −0.3208), 0.3974
Updated W_1, b_1	$0.5 + 0.0271, -0.3 - 0.0190$; $0.1 + 0.0271, -0.1 - 0.0190$	[[0.5271, −0.3190], [0.2, 0.4]], (0.1271, −0.1190)

After the step the prediction for the same example rises from 0.6051 to **0.6792** and the loss falls from 0.5023 to **0.3868**, as expected for a small gradient step. My NumPy backpropagation reproduces these gradients to within 10^{-17}.

6. Implementation and numerical gradient check

The network is implemented in NumPy (class MLP in Appendix A: forward pass, hand-derived backward pass, gradient-descent updates). Gradient check: central finite differences $(L(\theta + \varepsilon) - L(\theta - \varepsilon)) / 2\varepsilon$ with $\varepsilon = 10^{-5}$ on the full XOR batch (4 examples, random initial weights, seed 3). Maximum relative error $|g_a - g_n| / (|g_a| + |g_n|)$:

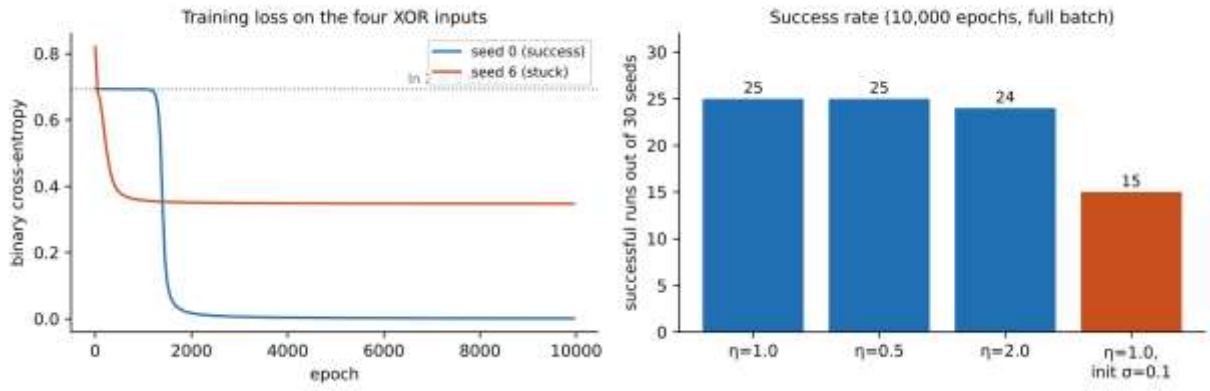
Parameter	Analytical gradient (backprop)	Numerical gradient	Max relative error
W1	0.0031, 0.0013, 0.0085, 0.0046	0.0031, 0.0013, 0.0085, 0.0046	1.7e-09
W2	-0.0583, -0.0097	-0.0583, -0.0097	2.5e-10
b1	0.0058, 0.0024	0.0058, 0.0024	4.3e-10
b2	-0.0744	-0.0744	2.1e-11

All errors are below 2×10^{-9}, so the analytical gradients are correct.

7. Training on the four input combinations

Full-batch gradient descent, $\eta = 1.0$, 10,000 epochs, Xavier-scale random initial weights, 30 different seeds.

Figure 4. Training the 2-2-1 sigmoid network on XOR



Result for a successful run (seed 0): final outputs for (0,0), (0,1), (1,0), (1,1) = 0.0010, 0.9990, 0.9986, 0.0009 with targets 0, 1, 1, 0; final loss 0.0011; 4/4 correct. The loss stays near ln 2 = 0.6931 for the first thousand epochs or more (a plateau caused by small, nearly symmetric hidden weights) and then falls quickly (Figure 4).

Investigation of the unsuccessful runs. With $\eta = 1.0$, 25 of 30 seeds learn XOR; seeds 6, 13, 21, 27 and 28 stop at loss 0.347 and only 2 of the 4 outputs are on the correct side (the other two sit at 0.50). For seed 6 the outputs are 0.0011, 0.4997, 0.9994, 0.5002 and the hidden-unit activations are (0,0): (0.017, 0.113), (0,1): (0.000, 1.000), (1,0): (0.858, 0.952), (1,1): (0.000, 1.000). The inputs (0,1) and (1,1), which need different labels (1 and 0), are mapped to the **same point in hidden space** (0, 1), where both hidden sigmoids are saturated; the output neuron cannot separate two identical hidden representations and the gradients through the saturated units are ≈ 0, so training cannot repair it (a vanishing-gradient local minimum). Changing η from 0.5 to 2.0 changes success only from 25 to 24 of 30, so the cause is the initial position, not the step size. With very small initial weights ($\sigma = 0.1$) only 15 of 30 runs escape the ln 2 plateau within 10,000 epochs, which shows that symmetric small initial weights give very slow learning. **Conclusion:** the 2–2–1 sigmoid network can represent XOR but whether it learns it depends on initialisation; the remedies are a larger hidden layer, better initial weights or restarts.

Q4 – Investigate Whether a Deeper Network Is a Better Design 6 Marks

1. Data, architectures and settings

Data: scikit-learn make_moons (1,000 observations, 2 features, noise = 0.25, random_state = 42); stratified split 600 training / 200 validation / 200 test (the same split for every model and seed); standardisation fitted on the 600 training observations. Because the noise is high the two crescents overlap, so even a perfect model cannot be 100 % accurate (the best accuracy is expected to be about 93–95 %).

Model	Hidden layers	Parameter count	Reason
Shallow S16	1 × 16 (ReLU)	$2\cdot16 + 16 + 16 + 1 =$ **65**	16 ReLU units give 16 linear pieces, enough to bend a boundary around two crescents
Deep D3×16	3 × 16 (ReLU)	$(2\cdot16+16) + 2(16\cdot16+16) + (16+1) =$ **609**	Same width as S16 so that only the number of layers differs
Control S152	1 × 152 (ReLU)	$2\cdot152 + 152 + 152 + 1 =$ **609**	Parameter-matched shallow network to separate the effect of depth from the effect of size

Common settings: sigmoid output, binary cross-entropy, He initialisation, Adam ($\eta = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$), mini-batch 32, 150 epochs, no regularisation, seeds 0, 1, 2 (they change initial weights and batch order only). The weights from the epoch with the lowest validation loss are kept; the test set is used only after the architecture has been selected on validation data.

2. Predictions before training

- **Representation:** all three can represent the crescent boundary; D3×16 can build it with fewer units per layer but there is no need for it in two dimensions.
- **Training behaviour:** D3×16 has more layers to propagate gradients through and more parameters, so I expect faster fitting of the training data (lower training loss) but a noisier validation curve.

- **Generalisation:** with 600 noisy observations the larger models will overfit sooner; I expect S16 to have equal or slightly lower validation loss, and no accuracy gain for the deep model.
- **Computational effort:** the deep model needs about 2× the time per epoch (three matrix products plus more bookkeeping), and S152 about 1.5×.

3. Results (mean ± standard deviation over 3 seeds)

Model (params)	Best epoch	Train loss	Val loss	Train acc	Val acc	Test acc	Time (s)
S16 (65)	36.7	0.1206 ± 0.0046	0.1672 ± 0.0017	0.963 ± 0.002	0.933 ± 0.006	0.932 ± 0.006	0.21
D3×16 (609)	10.7	0.1180 ± 0.0061	0.1768 ± 0.0044	0.958 ± 0.004	0.923 ± 0.002	0.925 ± 0.004	0.44
S152 (609)	13.0	0.1179 ± 0.0007	0.1740 ± 0.0020	0.962 ± 0.001	0.927 ± 0.002	0.935 ± 0.004	0.36

Model (params)	Train loss, epoch 150	Val loss, epoch 150	Train acc, epoch 150	Val acc, epoch 150
S16 (65)	0.1023 ± 0.0016	0.1831 ± 0.0036	0.961 ± 0.002	0.927 ± 0.002
D3×16 (609)	0.0720 ± 0.0022	0.3012 ± 0.0257	0.967 ± 0.001	0.923 ± 0.002
S152 (609)	0.0980 ± 0.0023	0.1907 ± 0.0040	0.963 ± 0.002	0.928 ± 0.006

(Time is the wall-clock time of the whole 150-epoch run on this computer, including the per-epoch evaluation; it was measured once per run, so differences below about 0.05 s are not meaningful.)

Figure 5. Learning curves of the shallow and deep networks

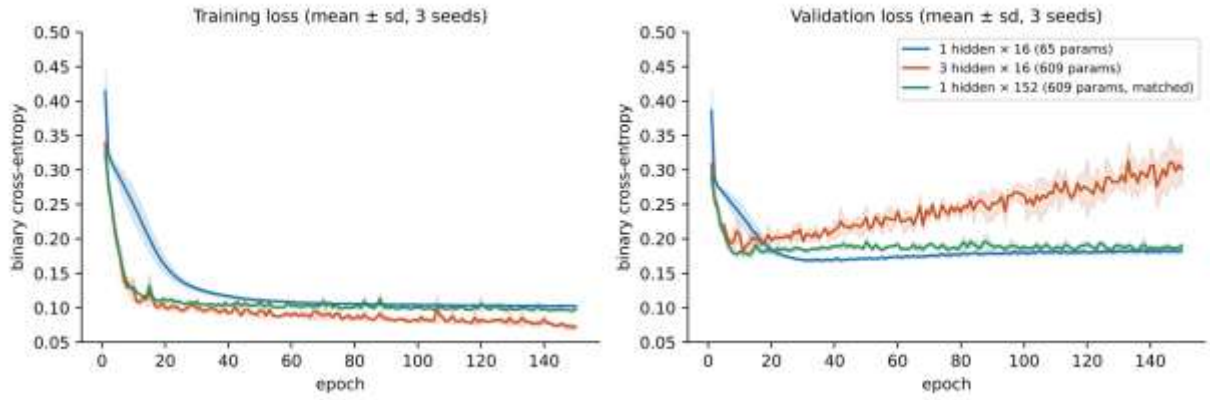
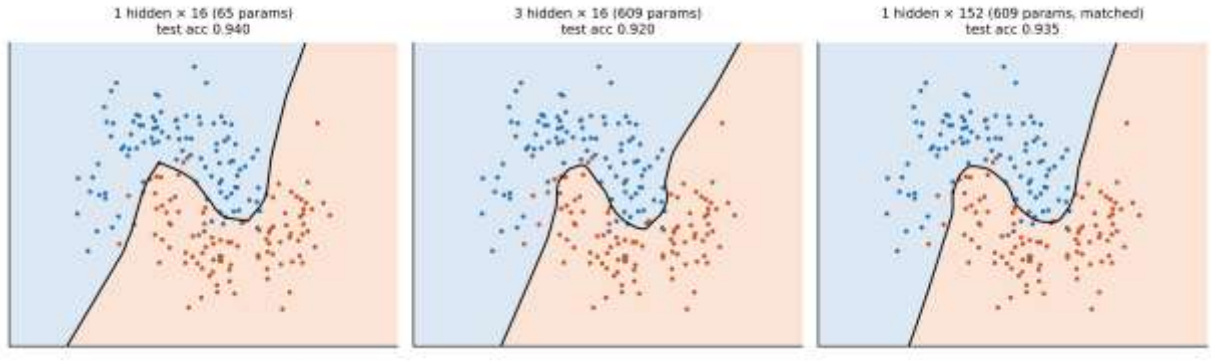


Figure 6. Decision boundaries (seed 0; points = test set, orange = defective class 1)



4. Interpretation

Selection on validation data: S16 has the lowest validation loss (0.1672) and the highest validation accuracy (0.933), so it is selected; D3×16 is the worst on validation (0.1768). The test accuracies (0.932, 0.925, 0.935) differ by less than one test observation per 100 and by less than their seed spread, so there is **no evidence that depth improves accuracy**. All three boundaries look alike

(Figure 6): they follow the two crescents and differ only in the overlap region. Predictions confirmed: the deep network reaches its best validation loss much earlier (epoch 11 against 37 for S16), then overfits – at epoch 150 its validation loss is 0.301 against 0.183 for S16 while its training loss is lower – and it costs about twice the training time (0.44 s against 0.21 s). Prediction not confirmed: the deep network was not noticeably noisier.

5. Design inquiry: recommendation

Recommend the shallow network S16 (one hidden layer of 16 ReLU units, 65 parameters): lowest validation loss, same test accuracy, 9× fewer parameters and about half the training time. D3×16 and S152 have the same 609 parameters; their validation losses (0.1768 and 0.1740) differ by only 0.003, comparable to their seed standard deviations (0.002–0.004), so the data cannot attribute any difference to depth rather than to size. Any improvement of the deep model over S16 would in any case be confounded with its 9× larger parameter count; the param-matched control shows that, at equal size, depth gives no benefit on this problem. Depth is expected to help when features need hierarchical composition (images, sequences), not for a smooth two-dimensional boundary.

Q5 – Diagnose Overfitting and Evaluate Regularization **7 Marks**

1. Experiment design

Limited-data set: 100 training observations (50 per class) drawn from the Q4 training partition with a fixed seed; the same 200 validation and 200 test observations as in Q4; standardisation fitted on the 100 training observations. **Baseline:** a comparatively large network $2 \to 128 \to 128 \to 1$ (ReLU, **17,025 parameters**, about 170 per training observation), Adam $\eta = 0.01$, mini-batch 16, 300 epochs, no regularisation, weights taken at epoch 300; 3 seeds (0, 1, 2). Interventions use the same network, data and settings.

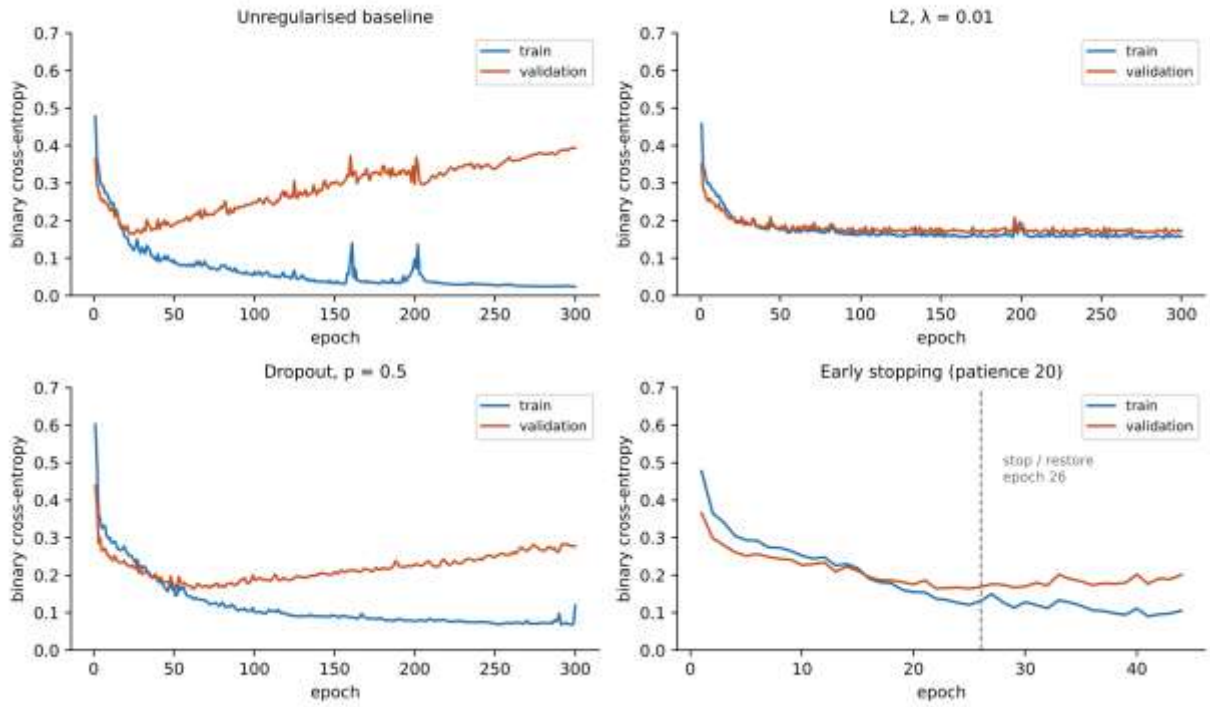
Prediction before training: training loss will approach 0 and training accuracy about 100 %, while validation loss will start rising after a few tens of epochs and validation accuracy will stay below training accuracy by at least 5–10 points (a clear overfitting gap).

2. Baseline and interventions (validation evidence, mean of 3 seeds)

Method	Setting	Train acc	Val acc	Gap	Train loss	Val loss	Epochs kept
Baseline	none	0.990	0.892	+0.098	0.0241	0.3938	300
L2	$\lambda = 0.0001$	0.990	0.895	+0.095	0.0279	0.2883	300
L2	$\lambda = 0.001$	0.977	0.920	+0.057	0.0690	0.1805	300
L2	$\lambda = 0.01$	0.957	0.920	+0.037	0.1578	0.1733	300
L2	$\lambda = 0.03$	0.833	0.888	-0.055	0.3493	0.2856	300
L2	$\lambda = 0.1$	0.837	0.883	-0.047	0.3882	0.3326	300
Dropout	$p = 0.2$	0.990	0.912	+0.078	0.0327	0.3773	300
Dropout	$p = 0.5$	0.950	0.925	+0.025	0.1198	0.2760	300
Dropout	$p = 0.8$	0.917	0.912	+0.005	0.1702	0.2345	300
Early stopping	patience 20	0.957	0.923	+0.033	0.1208	0.1593	26

Rules and strengths. L2: loss + $(\lambda/2)\Sigma W^2$ (weights only, not biases), λ chosen from $\{10^{-4}, 10^{-3}, 10^{-2}, 3\times10^{-2}, 10^{-1}\}$. Dropout: inverted dropout on both hidden layers, p chosen from {0.2, 0.5, 0.8}. Early stopping: stop when validation loss has not improved for **20 consecutive epochs** and restore the weights of the best epoch (a patience of 10 gave the same result). Within each method I pick the setting with the lowest **validation loss**: L2 $\lambda = 0.01$, dropout $p = 0.8$, early stopping (best epoch 26).

Figure 7. Training and validation loss with 100 labelled observations (mean of 3 seeds)



3. Analysis

Is there overfitting in the baseline? Yes. Training accuracy is 0.990 but validation accuracy only 0.892 (gap 0.098); training loss falls to 0.0241 while validation loss is 0.3938, 16 times larger, and the validation curve rises from about epoch 25 while the training curve keeps falling (Figure 7). The prediction was confirmed.

Most useful improvement: early stopping gives the lowest validation loss (0.1593, from 0.3938) and the smallest cost (it also trains for only 26 + 20 epochs instead of 300). L2 with $\lambda = 0.01$ is close (0.1733, accuracy 0.920, gap 0.037) but needs all 300 epochs; dropout reduces the gap (0.025 at $p = 0.5$) but leaves the validation loss much higher (0.276–0.235) because it keeps the net noisy at evaluation scale.

Underfitting: yes for strong L2: $\lambda = 0.03$ and 0.1 give training accuracy 0.833 and 0.837, below the validation accuracy (0.888, 0.883), a negative gap, high training loss (0.35–0.39) and test accuracy of only 0.82 and 0.81, which is the signature of high bias. Dropout $p = 0.8$ also raises the training loss to 0.170 and lowers the training accuracy to 0.917, a mild loss of fit.

How each intervention changes learning: L2 pulls all weights towards zero throughout training, so the decision boundary stays smoother; dropout injects noise so that hidden units cannot rely on each other, which slows the decrease of the training loss; early stopping does not change the objective but cuts the trajectory at the point where validation loss is lowest, before the network starts to memorise the noisy points.

Does higher training accuracy mean a better model? No. The baseline has the highest training accuracy (0.990) but the worst validation loss; the selected model has training accuracy 0.957 and the best validation loss. Training accuracy measures memorisation of 100 points, validation data measure generalisation.

4. Selection and final test evaluation

Preferred configuration chosen **on validation evidence only: early stopping** (patience 20, restore best epoch). Only now is the test set used (3 seeds, 200 test observations):

Model	Test accuracy	Test loss
Baseline (epoch 300)	0.938 ± 0.010	0.4431
Selected: early stopping	0.923 ± 0.002	0.1947

The test accuracy of the baseline (0.938) is not lower than that of the selected model (0.923): with 200 test observations one observation is 0.5 points and the standard error of an accuracy near 0.93 is about 1.8 points, so this 1.5-point difference is **not significant**. The test loss, which is sensitive to overconfident wrong predictions, is however much better for the selected model (see table), which confirms that the baseline is overfitted even when its accuracy looks acceptable.

Q6 – Engineer a Hyperparameter and Optimization Strategy **8 Marks**

1. Search plan (12 runs)

Data and partitions of Q4 (600/200/200), standardisation fitted on the training set; network 2 → w → 1 with ReLU; maximum **150 epochs**, stopping rule: stop after **20 epochs without improvement of validation loss**, keep the best epoch; one seed (42) per run because of the 12-run budget. The plan is a base configuration plus one-factor-at-a-time changes, with SGD and momentum compared at their own learning rates.

Run	Optimiser	η	Batch	Width	Change / purpose
R1	Adam	0.01	32	16	Base
R2	Adam	0.0001	32	16	LR: 0.01 → 0.0001 (too small)
R3	Adam	0.001	32	16	LR: 0.01 → 0.001
R4	Adam	0.1	32	16	LR: 0.01 → 0.1 (too large)
R5	Adam	0.01	8	16	Batch 32 → 8
R6	Adam	0.01	128	16	Batch 32 → 128
R7	Adam	0.01	32	4	Width 16 → 4
R8	Adam	0.01	32	64	Width 16 → 64
R9	SGD	0.1	32	16	Optimiser Adam → SGD (η = 0.1)
R10	SGD	0.01	32	16	SGD, η = 0.01
R11	Momentum	0.01	32	16	SGD + momentum 0.9, η = 0.01
R12	Momentum	0.1	32	16	SGD + momentum 0.9, η = 0.1

Budget allocation: 1 base run + 3 for the learning rate (the most influential hyperparameter for gradient methods, tested on both sides of the base, with one run per extreme to show too small and too large) + 2 for batch size + 2 for width + 4 for the optimiser (SGD at two learning rates and momentum at two learning rates, because SGD-type optimisers are far more sensitive to η than Adam). **Controlled comparisons:** R1 vs R2/R3/R4 (learning rate only); R1 vs R5/R6 (batch only); R1 vs R7/R8 (width only); R9 vs R10 (SGD learning rate only); R10 vs R11 (momentum only, same η); R9 vs R12 (momentum only, same η). The comparison R1 vs R9 changes both the optimiser and η and is therefore not a controlled comparison.

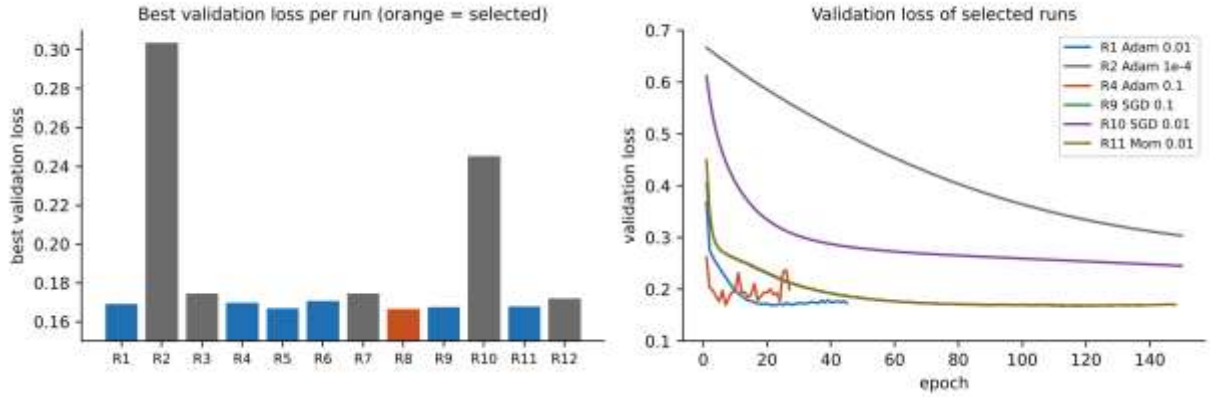
Predictions before experimentation: a learning rate that is too small (Adam 10^{-4}, SGD 0.01) learns slowly: validation loss still falling at epoch 150 and a higher final loss; a learning rate that is too large (Adam 0.1) gives an erratic, spiking loss curve, the best epoch very early and possibly

worse final loss or dead ReLUs; momentum and Adam will converge faster than plain SGD at the same η; width and batch size should have a small effect on a problem this simple. **Selection measure:** the lowest **validation binary cross-entropy** at the best epoch (accuracy on 200 points is too coarse); differences smaller than 0.002 (about the seed-to-seed standard deviation observed in Q4) are treated as ties and broken by lower training time.

2. Results

Run	Best val loss	Val acc	Best epoch / epochs run	Train loss	Time (s)	Signs of problems
R1	0.1692	0.930	25 / 45	0.1238	0.06	stable
R2	0.3035	0.905	150 / 150	0.3373	0.21	still improving at epoch 150 (slow)
R3	0.1746	0.930	150 / 150	0.1449	0.21	still improving at epoch 150 (slow)
R4	0.1696	0.925	7 / 27	0.1144	0.04	unstable (val-loss jump ×1.30)
R5	0.1670	0.935	14 / 34	0.1226	0.17	stable
R6	0.1705	0.925	52 / 72	0.1310	0.04	stable
R7	0.1744	0.940	46 / 66	0.1391	0.09	stable
R8	0.1667	0.930	21 / 41	0.1120	0.08	stable
R9	0.1674	0.935	118 / 138	0.1187	0.12	stable
R10	0.2453	0.900	150 / 150	0.2698	0.13	still improving at epoch 150 (slow)
R11	0.1680	0.930	128 / 148	0.1168	0.15	stable
R12	0.1721	0.930	8 / 28	0.1379	0.04	stable

Figure 8. Results of the 12-run search



3. Selection and analysis

Final configuration: R8 (Adam, η = 0.01, batch 32, one hidden layer of 64 ReLU units, 257 parameters): lowest validation loss (0.1667). Runs R5, R9 and R11 are within the 0.002 tie band (0.1670, 0.1674, 0.1680), so the selection also had to consider cost: R8 had the shortest training time among them (0.08 s, stopping at epoch 41) and I therefore keep it. R1 (0.1692) is just outside the band.

Strongest influences: (1) the learning rate: Adam 10^{-4} gives 0.3035 and SGD 0.01 gives 0.2453 against about 0.167–0.170 for well-tuned runs (the small-rate runs were still improving at epoch 150, as predicted); (2) the optimiser matters mainly through its effective step size: SGD with η = 0.1 (0.1674) and momentum 0.9 with η = 0.01 (0.1680) reach the same loss as Adam but need 118 and 128 epochs instead of 25, i.e. Adam is the most robust and fastest (best epoch 25), but not better at its best; momentum with a large step (R12, η = 0.1) was faster but marginally worse (0.1721). Adam with η = 0.1 (R4) worked here (0.1696) but its validation loss jumped by a factor of 1.3 in one epoch, the instability predicted for a large step; (3) batch size and width had little effect (0.1667–

0.1744 over widths 4–64 and batches 8–128), and the 4-unit net was slightly worse (0.1744) – a sign of mild underfitting.

Performance versus cost: R8 and R1 reach nearly the same loss; R8 has 4× the parameters (257 vs 65) and a higher cost per epoch but stops sooner. The cheapest acceptable alternative is R7 (17 parameters, 0.1744 val loss, 0.940 val acc): about 5 % worse in loss for 15× fewer parameters, which might be preferred on constrained hardware. Batch 8 (R5) took 0.17 s versus 0.04 s for batch 128 with the same quality: smaller batches cost time here without benefit.

Limits caused by the small budget: one seed per run (differences below about 0.002 are noise); only one-factor-at-a-time changes, so interactions (for example large learning rate with large batch) were not explored; the width and batch grids have only three levels; a single learning rate per family of SGD; and one dataset. **Why the test set must stay outside the tuning process:** if the test set influenced any choice, the best of many candidates would be selected partly for chance agreement with the test data, so the reported test score would be optimistically biased and no longer an unbiased estimate of future performance.

Test performance of the selected configuration R8 (reported once, not used to revise anything): test accuracy **0.940**, test loss **0.1626**.

Q7 – Diagnose, Redesign and Validate a Neural Network Solution 13 Marks

Solution selected for redesign: the Q5 baseline (2 → 128 → 128 → 1 network trained on 100 labelled observations). Honest note on the process: the redesign candidates were explored on seeds 0–4 using validation data only; the final comparison below uses fresh seeds 10–14 that played no part in the design, and the test set was used only at the very end.

A. Diagnose the learning or computational bottleneck (2 marks)

Evidence (Q5, mean of 3 seeds)	Observation
Training versus validation accuracy	0.990 versus 0.892: gap 0.098
Training versus validation loss	0.0241 versus 0.3938: validation loss 16× larger
Shape of the curves	Validation loss lowest at epoch ≈ 26 and rising afterwards while training loss keeps falling
Capacity versus data	17,025 parameters for 100 observations (≈ 170 per observation)
Needed capacity (Q4, Q6)	A 65-parameter network already reaches 0.93 validation accuracy on 600 observations, and a 17-parameter network reaches 0.94 (run R7)

Diagnosis: the bottleneck is **limited data combined with excessive model size** – the network has far more capacity than 100 noisy observations can constrain, so it fits the noise (variance error). It is not a representation problem (the model can represent the boundary), not a gradient problem (the training loss decreases smoothly without spikes), not a configuration problem (Adam with $\eta = 0.01$ trains stably) and the computational cost (0.85 s) is secondary, though it is also wasted on useless capacity.

B. Define the design objective (2 marks)

Item	Specification
Baseline (Q5)	Validation loss 0.394, generalisation gap (train acc − val acc) 0.098, 17,025 parameters, 300 epochs
Target	(1) Validation loss ≤ 0.165 (a reduction of at least 55 %); (2) accuracy gap ≤ 0.03; (3) validation accuracy ≥ 0.92; (4) at most 200 trainable parameters
Acceptable trade-offs	More training epochs to reach the best validation loss are acceptable if wall-clock time does not exceed the baseline; a small reduction in capacity is acceptable provided validation loss meets the target. The objective is judged on validation data only; the test set is not part of the objective.

C. Design the revised learning strategy (4 marks)

What changes and why. (1) **Right-size the network** to the information in 100 observations: replace 128–128 by a single hidden layer chosen on validation data from {4, 8, 16, 32} units – this attacks the diagnosed cause (excess capacity) directly instead of adding a regulariser to an oversized model. (2) **Keep early stopping** (patience 20, restore best epoch) as a safety net, because even a small network can overfit later in training. Everything else (Adam, $\eta = 0.01$, batch 16, ReLU, standardisation, partitions) is unchanged so that the comparison isolates the effect of capacity. Property of the original problem addressed: very small training sample with noisy class overlap and a simple, smooth boundary.

Exploration on validation data (seeds 0–4, mean ± sd of validation loss):

Candidate	Parameters	Val loss	Gap (train−val acc)	Decision
Baseline 128-128, no regularisation	17,025	0.3840 ± 0.0257	0.095	Diagnosed problem
128-128 + early stopping	17,025	0.1585 ± 0.0017	0.034	Large model kept
128-128 + input noise σ = 0.1 / 0.2 / 0.3 + ES	17,025	0.1580 / 0.1603 / 0.1705	0.025 / 0.021 / 0.019	No loss gain; σ = 0.3 hurts
128-128 + L2 10^{-3} + ES	17,025	0.1557 ± 0.0026	0.031	Best loss but same size
128-128 + noise 0.2 + L2 + ES (+ dropout 0.2)	17,025	0.1593 / 0.1634	0.021 / 0.018	No loss gain
Width 4 + ES	17	0.2072 ± 0.0461	−0.010	Underfits
Width 8 + ES	33	0.1750 ± 0.0140	0.022	Slightly worse
Width 16 + ES	**65**	**0.1568 ± 0.0049**	**0.015**	**Chosen (smallest within 0.002 of the best)**
Width 32 + ES	129	0.1566 ± 0.0024	0.028	No gain over 16
Width 16 + noise 0.1 / L2 10^{-4} + ES	65	0.1583 / 0.1576	0.021 / 0.015	No gain

Extra regularisers added nothing once capacity matched the data; the 4-unit net shows the new limitation (underfitting if capacity is cut too far).

```
REDESIGNED TRAINING WORKFLOW
 1. Split data: train (100 labelled), validation (200), test (200, locked)
 2. Fit standardiser on train only; apply to validation and test
 3. For width in {4, 8, 16, 32}:                # capacity search on validation data
      build network 2 -> width -> 1 (ReLU, sigmoid output), He init
      train with Adam (lr 0.01, batch 16), up to 300 epochs
      every epoch: compute validation loss; if best so far, store a copy of the weights
      stop if validation loss has not improved for 20 epochs
      record best validation loss
 4. Choose the smallest width whose validation loss is within 0.002 of the best
 5. Evaluate the chosen model once on the test set; deploy
```

Additional computation and memory: a capacity search of 4 short runs (≈ 0.4 s here) and a stored copy of the best weights; the final model needs about 260× less memory and fewer multiply-adds per prediction (65 versus 17,025 parameters). **New limitations:** with more data or a more complex boundary a width of 16 may underfit (width 4 already does here); more epochs are needed to reach the best validation loss (134 versus 27); the choice of width itself relies on a 200-observation validation set.

D. Predict before implementing (1 mark)

Based on Q4/Q6 (small networks fit this boundary) and on the exploration seeds, the redesigned network (2 → 16 → 1 with early stopping) **should**: (i) reach a validation loss within ±0.005 of the large network with early stopping (≈ 0.158) and below 0.165; (ii) show an accuracy gap ≤ 0.03; (iii) use about 260× fewer parameters; (iv) need many more epochs to reach its best validation loss (≈ 100–150 instead of ≈ 26) but not more wall-clock time. The **measure expected to improve** is the **validation loss and generalisation gap relative to the baseline**; I do **not** predict an improvement in loss relative to early stopping alone, only in size and gap.

E. Implement and validate the redesign (2 marks)

Same data partitions, same optimiser settings, **fresh seeds 10–14** (mean ± sd over 5 seeds). A = original baseline, B = best single Q5 intervention (early stopping on the large net), R = redesign.

Model	Parameters	Epochs kept	Train acc	Val acc	Gap	Val loss	Time (s)
A Baseline (128-128)	17,025	300	0.988	0.895	0.093	0.3637 ± 0.0394	0.89
B Early stopping (128-128)	17,025	27	0.958	0.919	0.039	0.1576 ± 0.0044	0.14
R Redesign (16 + ES)	65	134	0.954	0.933	0.021	0.1604 ± 0.0062	0.08

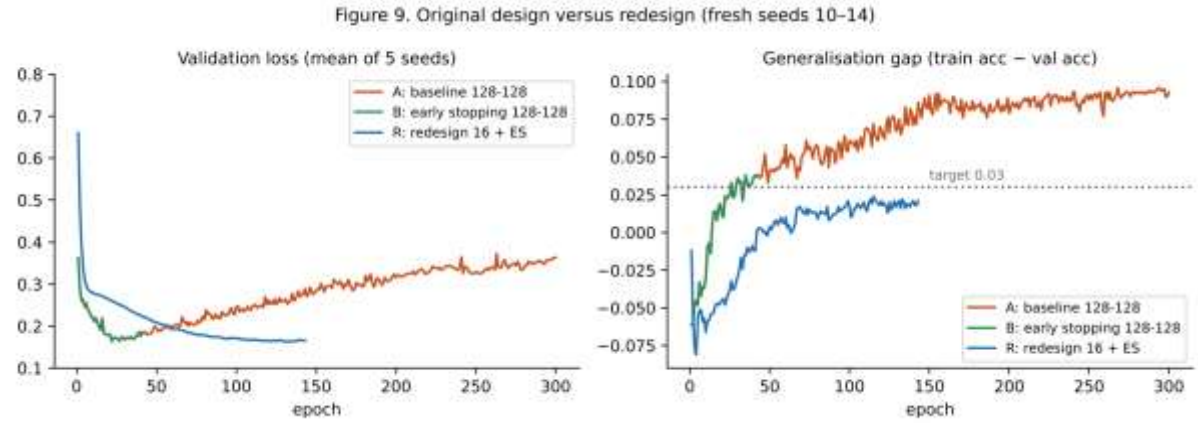


Figure 9. Original design versus redesign (fresh seeds 10–14)

Test set (used once, after the design was fixed):

Model	Test accuracy	Test loss
A Baseline	0.940 ± 0.005	0.4347
B Early stopping	0.933 ± 0.008	0.1873
R Redesign	0.916 ± 0.004	0.1829

F. Justify the final design (2 marks)

Objective / prediction	Target	Observed (R)	Met?
Validation loss	≤ 0.165	0.1604	Yes
Accuracy gap	≤ 0.03	0.021	Yes
Validation accuracy	≥ 0.92	0.933	Yes
Parameters	≤ 200	65	Yes
Time not above baseline	≤ 0.89 s	0.08 s	Yes
Loss within ±0.005 of ES on large net	0.153–0.163	0.1604 (ES: 0.1576)	Yes
More epochs to the best loss	≈ 100–150	134	Yes

Evaluation. The redesign achieved every objective and prediction on validation data: the validation loss fell from 0.364 (A, same seeds) to 0.160, the gap from 0.093 to 0.021, and the model is 262× smaller and about 11× faster to train than the baseline. **But the evidence also limits the claim:** compared with early stopping on the large network (B), the redesign gives no better validation loss (0.1604 versus 0.1576) – the main loss reduction comes from stopping at the right epoch, not from the new architecture – and its advantages over B are the smaller gap (0.021 versus 0.039), 262× fewer parameters and lower training time (0.08 s versus 0.14 s). On the test set the accuracy of R (0.916) is lower than that of A (0.940) and B (0.933); with 200 test observations the standard error is about 1.8 points, so the differences are not significant, but the redesign shows **no accuracy gain**; its test loss is the best (0.183 versus 0.187 for B and 0.435 for A). The part of the hypothesis that said a right-sized network would match, not beat, the regularised large network is therefore supported, and an accuracy benefit is not.

Final design decision: adopt the right-sized network with early stopping ($2 \rightarrow 16 \rightarrow 1$) because it meets the objective at a tiny fraction of the cost and shows the smallest generalisation gap. **Conditions under which the original (large) solution may remain preferable:** when considerably more labelled data become available (so that the 17,025 parameters are constrained), when the true boundary is more complex than two crescents (the 16-unit net could underfit, as the 4-unit net did), or when the cost of choosing the width is unacceptable; in these cases the large network with early stopping (B) is the safer choice. **Remaining limitation and monitoring:** the evaluation used one synthetic data set and 200 validation observations; after deployment I would monitor validation loss and accuracy gap on fresh labelled batches and retrain with a larger width search when the loss rises by more than 0.01 or the gap exceeds 0.03.

APPENDIX A – Source code (nn.py)

Experiment scripts (q123.py, q4.py, q5.py, q6.py, q7.py, q7b.py, q7c.py, figs.py) are supplied in the accompanying zip file; all use fixed seeds. Run with: python3 q123.py ; python3 q4.py ; python3 q5.py ; python3 q6.py ; python3 q7.py ; python3 q7c.py.

```
"""ITA06 Assignment 2 (CO2) - NumPy neural-network toolkit written from scratch.
Gradients are derived by hand (no automatic differentiation). Seeds are fixed for reproducibility."""
import numpy as np, time


def sigmoid(z): return 1.0/(1.0+np.exp(-np.clip(z,-60,60)))
def bce(y,a,eps=1e-12):
    a=np.clip(a,eps,1-eps); return float(-np.mean(y*np.log(a)+(1-y)*np.log(1-a)))
ACT={'relu':(lambda z:np.maximum(0,z),lambda z,a:(z>0).astype(float)),
     'tanh':(np.tanh,lambda z,a:1-a**2),
     'sigmoid':(sigmoid,lambda z,a:a*(1-a))}


class MLP:
    def __init__(s,sizes,hidden='relu',seed=0,init=None):
        s.sizes=sizes; s.hidden=hidden; rng=np.random.default_rng(seed); s.W=[]; s.b=[]
        for i in range(len(sizes)-1):
            fan_in,fan_out=sizes[i],sizes[i+1]
            if init is not None: sc=init
            elif hidden=='relu': sc=np.sqrt(2.0/fan_in)       # He initialisation
            else: sc=np.sqrt(2.0/(fan_in+fan_out))            # Xavier initialisation
            s.W.append(rng.normal(0,sc,(fan_in,fan_out))); s.b.append(np.zeros(fan_out))
    def n_params(s): return sum(w.size+b.size for w,b in zip(s.W,s.b))
    def forward(s,X,p_drop=0.0,rng=None):
        f=ACT[s.hidden][0]; A=[X]; Z=[None]; M=[None]; a=X
        for i,(W,b) in enumerate(zip(s.W,s.b)):
            z=a@W+b; last=(i==len(s.W)-1)
            a=sigmoid(z) if last else f(z)
            m=None
            if (not last) and p_drop>0:                       # inverted dropout on hidden layers
                m=(rng.random(a.shape)>=p_drop)/(1-p_drop); a=a*m
            Z.append(z); A.append(a); M.append(m)
        return A,Z,M
    def predict_proba(s,X): return s.forward(X)[0][-1].ravel()
    def backward(s,A,Z,M,y,l2=0.0):
        """Gradients of mean BCE (+ (l2/2)*sum W^2). Output delta = (a - y)/m for sigmoid + BCE."""
        m=len(y); g=ACT[s.hidden][1]; L=len(s.W); dW=[None]*L; db=[None]*L
        d=(A[-1]-y.reshape(-1,1))/m
        for i in reversed(range(L)):
            dW[i]=A[i].T@d+l2*s.W[i]; db[i]=d.sum(0)
            if i>0:
                d=(d@s.W[i].T)
                if M[i] is not None: d=d*M[i]
                d=d*g(Z[i],A[i])
        return dW,db
    def get(s): return [w.copy() for w in s.W],[b.copy() for b in s.b]
    def set(s,st): s.W=[w.copy() for w in st[0]]; s.b=[b.copy() for b in st[1]]


class Opt:
    def __init__(s,name,lr,params,beta=0.9,b1=0.9,b2=0.999,eps=1e-8):
        s.name,s.lr,s.beta,s.b1,s.b2,s.eps,s.t=name,lr,beta,b1,b2,eps,0
        s.v=[np.zeros_like(p) for p in params]; s.m=[np.zeros_like(p) for p in params]
    def step(s,params,grads):
        s.t+=1
        for i,(p,g) in enumerate(zip(params,grads)):
            if s.name=='sgd': p-=s.lr*g
            elif s.name=='momentum': s.v[i]=s.beta*s.v[i]+g; p-=s.lr*s.v[i]
            else:                                             # Adam
                s.m[i]=s.b1*s.m[i]+(1-s.b1)*g; s.v[i]=s.b2*s.v[i]+(1-s.b2)*g*g
                mh=s.m[i]/(1-s.b1**s.t); vh=s.v[i]/(1-s.b2**s.t); p-=s.lr*mh/(np.sqrt(vh)+s.eps)


def fit(net,Xtr,ytr,Xva,yva,epochs=150,batch=32,lr=0.01,opt='adam',l2=0.0,dropout=0.0,
        patience=None,seed=0,beta=0.9,Xte=None,yte=None,restore_best=True,noise=0.0):
    """Mini-batch training. History is measured in evaluation mode (no dropout, no L2 term).
    Keeps the weights with the lowest validation loss (restore_best)."""
    rng=np.random.default_rng(seed); params=net.W+net.b; o=Opt(opt,lr,params,beta=beta)
    H={k:[] for k in ['tr_loss','va_loss','tr_acc','va_acc']}; best=(np.inf,None,0); bad=0; t0=time.perf_counter(); stop=epochs
    n=len(ytr)
    for ep in range(1,epochs+1):
        idx=rng.permutation(n)
        for k in range(0,n,batch):
            j=idx[k:k+batch]; Xb=Xtr[j]+(rng.normal(0,noise,Xtr[j].shape) if noise>0 else 0.0); A,Z,M=net.forward(Xb,dropout,rng)
            dW,db=net.backward(A,Z,M,ytr[j],l2); o.step(net.W+net.b,dW+db)
        ptr=net.predict_proba(Xtr); pva=net.predict_proba(Xva)
        H['tr_loss'].append(bce(ytr,ptr)); H['va_loss'].append(bce(yva,pva))
        H['tr_acc'].append(float(np.mean((ptr>=.5)==ytr))); H['va_acc'].append(float(np.mean((pva>=.5)==yva)))
        if not np.isfinite(H['va_loss'][-1]): break
        if H['va_loss'][-1]<best[0]-1e-9: best=(H['va_loss'][-1],net.get(),ep); bad=0
        else: bad+=1
```

```
        if patience and bad>=patience: stop=ep; break
    t=time.perf_counter()-t0
    if restore_best and best[1] is not None: net.set(best[1])
    res=dict(H=H,best_epoch=best[2],stopped=len(H['va_loss']),time=t,params=net.n_params())
    k=(best[2] if restore_best else len(H['va_loss']))-1                    # epoch whose weights are kept
    res['best_va_loss']=best[0]; res['kept_epoch']=k+1; res['va_loss']=H['va_loss'][k]; res['tr_loss']=H['tr_loss'][k]
    res['va_acc']=H['va_acc'][k]; res['tr_acc']=H['tr_acc'][k]
    res['best_va_acc']=H['va_acc'][best[2]-1] if best[2] else None; res['best_tr_acc']=H['tr_acc'][best[2]-1] if best[2] else None; res['best_tr_loss']=H['tr_loss'][best[2]-1] if
best[2] else None
    if Xte is not None: res['te_acc']=float(np.mean((net.predict_proba(Xte)>=.5)==yte)); res['te_loss']=bce(yte,net.predict_proba(Xte))
    return res


def standardise(Xtr,*others):
    mu=Xtr.mean(0); sd=Xtr.std(0)+1e-12
    return [(X-mu)/sd for X in (Xtr,)+others]


def make_data(seed=42,n=1000,noise=0.25):
    from sklearn.datasets import make_moons
    X,y=make_moons(n_samples=n,noise=noise,random_state=seed)
    rng=np.random.default_rng(seed); tr,va,te=[],[],[]
    for c in (0,1):                                    # stratified 60/20/20 split
        idx=rng.permutation(np.where(y==c)[0]); a=int(.6*len(idx)); b=int(.8*len(idx))
        tr+=list(idx[:a]); va+=list(idx[a:b]); te+=list(idx[b:])
    tr,va,te=[np.array(rng.permutation(i)) for i in (tr,va,te)]
    return X[tr],y[tr].astype(float),X[va],y[va].astype(float),X[te],y[te].astype(float)
```